SOFTWARE REQUIREMENTS SPECIFICATION

GuardianAI

Real-Time Automated Fall Detection and Alert System

For Elderly and Patient Safety in Low-Resource Environments

Group ID	FYP-2026-G6
Document	Software Requirements Specification (SRS)
Version	**3.0** (supersedes V2)
Deliverable	FYP-I Checkpoint-2
Standard	Adapted from IEEE Std 830-1998 / ISO-IEC-IEEE 29148
Programme	BS Computer Science

Team Member	Roll No	Role
Kashan M. Ashraf	BSCS-01007	Video Processing + Testing
Umer Ahmed	BSCS-01011	AI Model + Development
Imran	BSCS-01014	UI Design + Documentation

Repository: FYP-2026-G6-RealTimeAutomatedFallDetectionAndAlertSystem

Prepared for review and approval by the Faculty Supervisor

Document Control

Revision History

Version	Stage	Author	Summary of Change
1.0	Proposal	FYP-2026-G6	Initial requirements captured from the approved project proposal.
2.0	Checkpoint-1	FYP-2026-G6	Expanded functional requirements FR-1 to FR-5, external interface definitions, and an initial architecture sketch.
3.0	**Checkpoint-2**	FYP-2026-G6	**Current revision.** Corrected factual inconsistencies carried over from V2; replaced the unrendered architecture source block with proper diagrams; introduced formal requirement identifiers with priority and verification method; added a system context diagram, use-case model, two previously undocumented requirement groups (FR-6 operator control, FR-7 training and data management), structured non-functional requirement IDs, acceptance criteria, and forward traceability to the Software Design Document. A full itemised change log appears in Section 9.

Document Approval

Role	Name	Signature / Date
Group Lead	Umer Ahmed	
Faculty Supervisor		

Related Documents

Document	Location	Relationship
Software Design Document (SDD) v1.0	`/docs`	Child document. Specifies *how* the requirements in this SRS are realised. Section 8 provides forward traceability into it.
SRS v2	`/docs`	Superseded by this revision. Retained for audit purposes.
FYP Proposal	`/reports`	Source of the original problem statement and objectives.
Implementation Evidence	`/evidence`	Verification artefacts referenced by the acceptance criteria in Section 7.

Table of Contents

1. Introduction

1.1 Purpose

This document specifies the software requirements for **GuardianAI**, a real-time automated fall detection and alert system. It defines *what* the system must do, the constraints under which it must operate, and the criteria by which each requirement is judged satisfied. It is the authoritative reference against which the implementation is evaluated.

GuardianAI addresses a specific problem: in Pakistan, elderly care and patient monitoring are constrained by staff shortages and the absence of smart healthcare infrastructure. Falls in homes and hospitals frequently go unnoticed, and the delay between a fall and its discovery is a primary determinant of injury severity. This system detects falls in real time from ordinary camera feeds, processes video entirely on local hardware, and alerts caregivers immediately — without requiring wearable devices or costly specialised sensors.

1.2 Scope

The GuardianAI system encompasses the following capabilities:

- Real-time video ingestion from local webcams, USB cameras, IP/RTSP streams, or recorded video files.
- Human pose estimation and skeletal joint tracking using YOLOv8-Pose.
- Derivation of spatial and temporal biomechanical features — body orientation angle, head drop speed, ground proximity, and postural aspect ratio.
- A hybrid decision engine fusing an explainable rule-based state machine with a temporal deep learning classifier (CNN-LSTM, with LSTM and Transformer alternatives).
- Multi-channel notification: console output, on-screen popup, audible buzzer, native operating-system notification, optional SMTP e-mail, and persistent JSON event logging with JPEG evidence capture.
- An interactive web monitoring dashboard presenting a live annotated video feed, real-time telemetry, system health indicators, and an incident history.
- Operator controls for pausing the feed, muting alerts, and resetting tracking state.
- An offline training subsystem for dataset preprocessing, model training, evaluation, and ONNX export.

Explicitly out of scope for this release: clinical certification or diagnostic claims; wearable or accelerometer-based sensing; hospital information-system integration; native mobile applications; cloud or multi-site deployment; and identification of individuals by name or biometric identity.

1.3 Definitions, Acronyms and Abbreviations

Term	Definition
YOLOv8-Pose	You Only Look Once, version 8, pose variant — a single-stage neural network that detects persons and regresses 17 skeletal keypoints in one pass. **The pose estimation technology used by this system.**
MediaPipe	An open-source Google framework for multi-modal machine learning pipelines. *Historical only:* evaluated during Checkpoint-1 and replaced by YOLOv8-Pose. Retained in this glossary because it appears in earlier documents and in the project's development history.
Keypoint	A detected body joint expressed as `(x, y, confidence)`.
Critical keypoints	The 13 of 17 YOLO joints retained for fall analysis; the four facial landmarks are discarded.
CNN-LSTM	A hybrid network in which a 1-D convolutional front-end extracts local motion patterns and an LSTM models longer-range temporal progression.
Sequence buffer	A 30-frame sliding window of feature vectors forming one input sample to the temporal classifier.
Decision fusion	Weighted combination of the rule-based verdict and the neural verdict into a single classification.
Temporal smoothing	A majority-vote filter over recent frames that suppresses single-frame flicker.
Body orientation angle	The angle of the vector from the nose to the hip midpoint. Approximately 90° upright; approaching 0° or 180° when horizontal.
Ground proximity	Condition in which the head occupies the lower 40% of the frame.
XAI	Explainable Artificial Intelligence — the property that an automated decision can be traced to human-interpretable evidence.
MJPEG	Motion JPEG; a stream of independent JPEG images delivered over a single HTTP response.
HUD	Heads-Up Display — the telemetry overlay drawn onto the annotated video frame.
FPS	Frames Per Second.
SMTP	Simple Mail Transfer Protocol.
Edge deployment	Executing inference on the machine attached to the camera rather than transmitting video to a remote server.
MoSCoW	Prioritisation scheme: Must have, Should have, Could have, Won't have (this release).

1.4 References

1. Ultralytics YOLOv8 Documentation — `https://docs.ultralytics.com` *(primary pose estimation technology)*.
2. PyTorch Documentation — `https://pytorch.org/docs`.
3. Flask Documentation — `https://flask.palletsprojects.com`.
4. OpenCV Documentation — `https://docs.opencv.org`.
5. IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*.

6. ISO/IEC/IEEE 29148:2018, *Systems and software engineering — Life cycle processes — Requirements engineering*.
7. GuardianAI Software Design Document v1.0, FYP-2026-G6.
8. MediaPipe Pose Landmark Detection — `https://developers.google.com/mediapipe` *(historical reference; superseded)*.

1.5 Document Conventions

This revision introduces a formal convention for stating requirements. Every requirement carries a unique identifier, a priority, and a verification method.

Requirement identifiers

Prefix	Meaning
`FR-n.m`	Functional requirement, group *n*, item *m*.
`NFR-X.m`	Non-functional requirement, category *X* (P performance, R reliability, S safety, SEC security, U usability, M maintainability).
`UC-n`	Use case.

Priority (MoSCoW)

Level	Meaning
MUST	Mandatory for FYP-1. The system is not acceptable without it.
SHOULD	Important and expected, but the system remains demonstrable if it degrades.
COULD	Desirable enhancement; implemented if time permits.
WON'T	Explicitly deferred to FYP-2. Recorded so scope boundaries are unambiguous.

Verification method

Code	Meaning
T	Test — executed against defined inputs with a measurable pass condition.
D	Demonstration — observed live during the prototype demonstration.
I	Inspection — verified by examining source code, configuration or output artefacts.
A	Analysis — verified by measurement and reasoning, e.g. sustained frame-rate profiling.

The keywords **shall** (mandatory), **should** (recommended) and **may** (optional) are used consistently in the sense defined by IEEE Std 830-1998.

2. Overall Description

2.1 Product Perspective

GuardianAI is a **self-contained, edge-deployable monitoring application**. It is not a component of a larger product and has no mandatory dependency on any external service. It executes on a single commodity computer attached to a camera, performs all inference locally, and exposes a lightweight web interface over the local network.

This architecture is a direct consequence of the problem domain. Low-resource care environments cannot be assumed to have reliable internet connectivity, budget for cloud inference, or the legal and ethical clearance to transmit continuous video of vulnerable individuals off-site. Local processing resolves all three constraints simultaneously.

2.2 System Context

Figure 2.1 shows GuardianAI as a single system together with every external entity it exchanges data with.

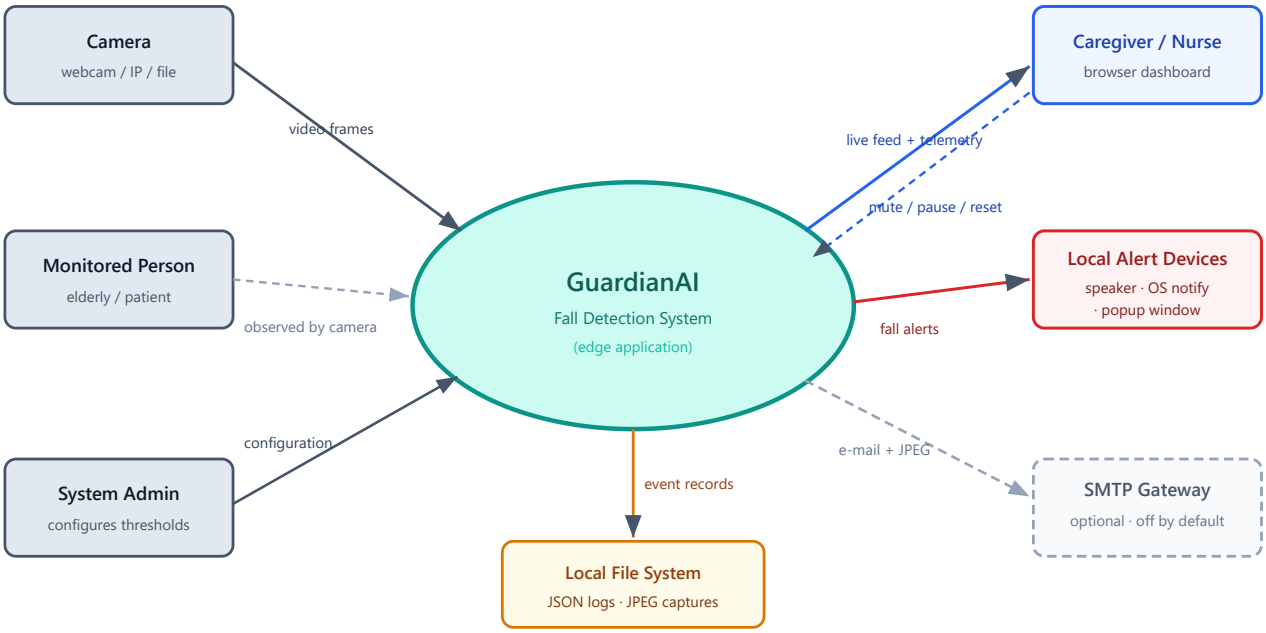

Figure 2.1 — System context diagram. Solid arrows are mandatory data flows; dashed arrows are optional or indirect. Raw video enters but never leaves the system.

2.3 Product Functions

Function	Description
Pose detection and tracking	Detect persons in the camera field of view and extract 17 skeletal joint coordinates each, retaining the 13 relevant to postural analysis.
Biomechanical assessment	Continuously measure body tilt angle, head drop speed and ground proximity, and expose these values for inspection.
Temporal AI classification	Analyse a 30-frame window (approximately one second at 30 FPS) and classify activity as `normal`, `warning` or `fall`.
Decision fusion	Combine heuristic and neural verdicts, biasing toward escalation when the interpretable rules detect greater severity.
Multi-channel alerting	Raise audible, visual, on-screen, operating-system and optional e-mail notifications, subject to a cooldown that prevents flooding.
Evidence logging	Persist every alert as a structured JSON record accompanied by a single JPEG frame.
Live monitoring dashboard	Serve an annotated video stream with real-time telemetry, system health indicators and an incident history.
Operator control	Allow the operator to pause the feed, mute alerts and reset tracking state without restarting the application.
Model training	Preprocess video datasets, train the temporal classifier, evaluate it, and export to ONNX.

2.4 User Classes and Characteristics

User class	Technical skill	Frequency	Needs and expectations
Caregiver / Nurse	Low — general computer literacy only	Continuous during shift	Must understand system state at a glance from across a room. Requires unambiguous alerts and the ability to silence them quickly without disabling monitoring. Must never be required to interpret technical output.
System Administrator	Moderate to high	Installation and occasional tuning	Installs dependencies, selects the camera source, adjusts detection thresholds for the room geometry, and optionally configures SMTP credentials.
Developer / Researcher	High	Development phase	Trains and evaluates models, extends the feature set, and inspects logs for accuracy analysis.
Monitored Person	None — passive	Continuous	Not a system operator. Requires that monitoring be non-intrusive, wearable-free, and privacy-preserving.

2.5 Use-Case Model

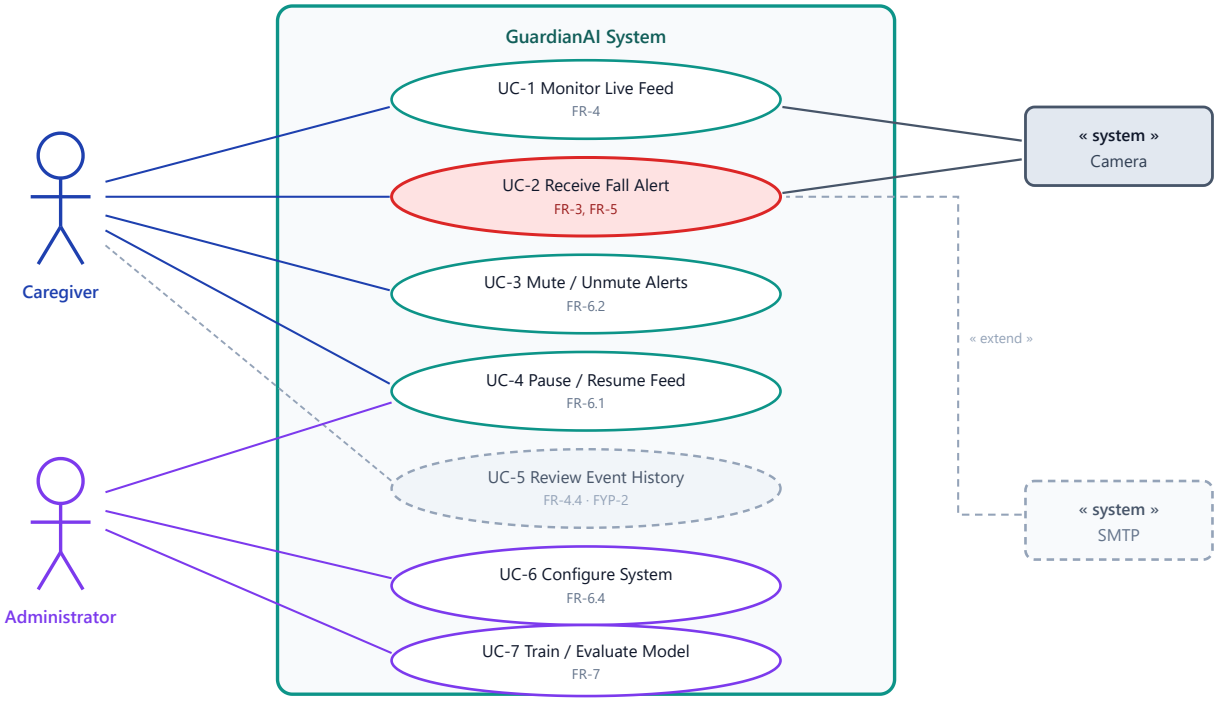

Figure 2.2 — Use-case model. The greyed, dashed use case (UC-5) is specified in this document but deferred to FYP-2.

UC-2 — Receive Fall Alert (primary use case, expanded)

Field	Detail
Actor	Caregiver (primary), Camera (supporting)
Precondition	System is running, camera is connected, monitored person is within the field of view.
Trigger	The monitored person falls.
Main flow	1. Camera delivers frames. 2. System extracts keypoints and computes biomechanical features. 3. Rule engine and neural classifier each produce a verdict. 4. Verdicts are fused and temporally smoothed. 5. Status resolves to `fall` with confidence above threshold. 6. System raises buzzer, popup, OS notification and dashboard overlay. 7. System writes a JSON event record and a JPEG capture. 8. Caregiver observes the alert and responds.
Alternative flow	5a. Confidence below threshold → status remains `warning`; dashboard reflects it, no alert is raised. 6a. Alerts are muted → buzzer, popup and OS notification are suppressed; logging and dashboard update continue. 6b. Cooldown active → alert suppressed to prevent flooding.
Exception flow	2a. Person not detected for more than 10 consecutive frames → tracking reported lost; status returns to `normal`. 1a. Camera read fails → system attempts reconnection; dashboard shows *No Camera*.
Postcondition	Alert delivered through all enabled channels; event persisted with evidence frame.

2.6 Operating Environment

Element	Requirement
Operating system	Windows 10 or 11 (primary, tested). The codebase is cross-platform; the buzzer degrades to terminal bells on Linux and macOS.
Runtime	Python 3.8 or later.
Processor	x86-64 CPU. A CUDA-capable GPU is detected and used automatically when present but is not required.
Memory	8 GB RAM recommended.
Camera	Any device enumerable by OpenCV, at 640 × 480 or higher.
Browser	Any modern browser supporting MJPEG in an `<img>` element.
Network	Local network only. Internet access is not required for operation.

2.7 Design and Implementation Constraints

Constraint	Rationale and consequence
Real-time latency	A fall must be recognised within 1–2 seconds of occurring, which bounds the temporal window and forbids blocking operations in the frame loop.
Explainability	Clinical settings require an auditable reason for every alert, so a purely black-box classifier is unacceptable; interpretable rules must remain a first-class decision path.
Privacy	Continuous video of vulnerable people is highly sensitive; the system must not record video.
Affordability	The target environment cannot fund specialised hardware, so the design is limited to a single ordinary camera and commodity computing.
Graceful degradation	An unattended safety system must not fail hard; every external dependency requires a defined fallback.
Academic scope	FYP-1 targets a demonstrable MVP, so flat-file persistence is acceptable and authentication is deferred.

2.8 Assumptions and Dependencies

Assumption or dependency	Impact if invalid
Camera field of view is unobstructed and covers the monitored area	Falls outside the field of view cannot be detected. Mitigated in part by 10-frame occlusion tolerance.
Lighting is sufficient for body-outline detection	Pose confidence drops below threshold and detection fails. No low-light enhancement stage exists.
The subject's hips and head are within frame	The orientation angle becomes unreliable; this is a known accuracy limitation at close camera range.
Python dependencies (PyTorch, Ultralytics, OpenCV, Flask) are installable	System cannot run. Dependencies are declared with lower bounds in `requirements.txt`.
A caregiver is reachable by at least one alert channel	Alerts are raised but unobserved. Multiple independent channels mitigate this.
SMTP credentials are supplied if e-mail alerting is enabled	E-mail channel silently disabled; all other channels unaffected.

3. Functional Requirements

Each requirement below states a single verifiable capability, together with its priority (MoSCoW) and verification method (T/D/I/A) as defined in Section 1.5.

3.1 FR-1 — Pose Estimation Pipeline

ID	Requirement	Priority	Verif.	Acceptance condition
FR-1.1	The system **shall** use the YOLOv8-Pose (nano) model to extract 17 skeletal keypoints from every processed video frame, discarding detections scoring below the configured confidence threshold (default 0.5).	MUST	T, I	With a person in frame, keypoints are returned; with an empty scene, none are.
FR-1.2	The system **shall** reduce the 17 detected keypoints to the 13 designated critical keypoints, discarding the four facial landmarks (eyes and ears).	MUST	I	Feature arrays have shape `(13, 3)`.
FR-1.3	The system **shall** normalise keypoint coordinates to the range `[0, 1]` against frame width and height, so downstream logic is independent of camera resolution.	MUST	I	All coordinate values lie within `[0, 1]` regardless of capture resolution.
FR-1.4	The system **shall** superimpose a skeleton overlay on the video stream, drawing 14 limb connections and joint markers, omitting any joint whose confidence is below 0.4.	SHOULD	D	Skeleton is visible and tracks the subject in the dashboard feed.
FR-1.5	Where a subject becomes temporarily undetected, the system **shall** retain the last known keypoints for up to 10 consecutive frames before declaring tracking lost.	MUST	T	Brief occlusion does not reset the temporal buffer or clear the current status.
FR-1.6	The system **should** support simultaneous tracking of up to 5 persons, maintaining independent keypoint buffers and detection state per subject.	WON'T (FYP-2)	D	Capability exists as `MultiPersonPoseEstimator` and is demonstrable standalone; integration into the live dashboard pipeline is deferred to FYP-2.

> **Scope correction from V2.** SRS v2 stated multi-person tracking as an active requirement and the project README listed it as complete. In fact only the single-subject path is wired into the live pipeline. FR-1.6 is therefore reclassified as WON'T (this release) so the delivered scope is stated accurately.

3.2 FR-2 — Biomechanical Feature Engineering

ID	Requirement	Priority	Verif.	Acceptance condition
FR-2.1	The system **shall** compute the body orientation angle as the absolute angle of the vector from the nose keypoint to the **midpoint of the left and right hip keypoints**: `angle = |degrees(arctan2(y_hip − y_nose, x_hip − x_nose))|`	MUST	T, I	Reports approximately 90° for an upright subject and approaches 0° or 180° for a horizontal subject.
FR-2.2	The system **shall** compute head drop speed as the frame-to-frame vertical displacement of the head keypoint, in pixels per frame against a virtual 640 × 480 reference frame: `speed = y_head(t) − y_head(t−1)`	MUST	T	Positive during downward motion, near zero when stationary.
FR-2.3	The system **shall** evaluate ground proximity as true when the head coordinate falls below 60% of the virtual frame height (`y > 288` px of 480).	MUST	T	Predicate is true when the subject is at floor level, false when standing.
FR-2.4	The system **shall** accumulate a rolling buffer of 30 frames — **approximately one second at 30 FPS** — as the input window for temporal classification, using a stride of 5 frames when generating training sequences.	MUST	I	Buffer reaches exactly 30 entries before neural inference is attempted.
FR-2.5	Where a frame contains no detected person, the system **shall** append a zero-valued feature vector rather than skipping the frame, preserving the real-time spacing of the temporal window.	MUST	I	Buffer length advances at a constant rate irrespective of detection gaps.
FR-2.6	The system **shall** encode each frame as a 51-dimensional feature vector comprising 39 flattened raw keypoint values and 12 engineered feature slots (5 active, 7 reserved).	MUST	I	Vector length is exactly 51 and matches `ModelConfig.input_size`.
FR-2.7	The system **should** apply the feature standardisation statistics computed during training to live inference input.	SHOULD	I	*Partially met.* Standardisation exists in the training path; it is not yet applied at live inference. Tracked as a known train/serve skew for FYP-2.
FR-2.8	The system **should** disregard the orientation angle when the contributing hip keypoints fall below a minimum confidence, holding the previous valid value instead.	SHOULD	T	*Not yet met.* New requirement raised at Checkpoint-2 in response to observed false `warning` states at close camera range (see Section 9).

3.3 FR-3 — Hybrid Decision Engine

ID	Requirement	Priority	Verif.	Acceptance condition
FR-3.1	The system **shall** apply a rule-based state machine classifying each frame as `normal`, `warning` or `fall` using the configured physical thresholds: • `normal` — angle ≥ 70° and no rapid descent. • `warning` — angle < 70°, or a lying posture sustained for ≥ 5 frames. • `fall` — rapid descent (> 20 px/frame) sustained for ≥ 2 frames, **confirmed by** ground proximity and an orientation angle < 50°.	MUST	T, D	*Partially met.* The `normal` and `warning` rules behave as specified. The `fall` rule currently triggers on the descent condition alone, without the ground-proximity and angle confirmation this requirement mandates. See the defect note below.
FR-3.2	The system **shall** classify 30-frame sequences using a CNN-LSTM temporal neural network producing confidence scores across the three classes.	MUST	T, I	Checkpoint loads and `/health` reports `model_loaded: true`.
FR-3.3	The system **shall** fuse rule and neural verdicts using a default neural weight of 0.7, and **shall** prioritise the rule verdict where it indicates greater severity with confidence above 0.8.	MUST	I	Fusion follows the specified branch logic; severity escalation is preserved.
FR-3.4	Where no trained checkpoint is available, the system **shall** operate on rule-based classification alone without raising an unhandled exception.	MUST	T	Renaming the checkpoint file yields a warning and continued operation.
FR-3.5	The system **shall** apply temporal smoothing over the last 5 frames, emitting `fall` only where at least 3 of 5 frames classify as `fall`, and `warning` where at least 2 of 5 classify as `warning`.	MUST	T, I	Single-frame anomalies do not alter the reported status.
FR-3.6	The system **shall** raise an alert only where the smoothed status is `fall` and fused confidence exceeds 0.7.	MUST	T	Low-confidence detections update the dashboard without triggering alerts.
FR-3.7	The system **shall** expose the raw biomechanical evidence (angle, speed) underlying every classification through both the video HUD and the status API.	MUST	D, I	Angle and speed are visible on the overlay and present in `/status`.

Open defect against FR-3.1 (highest-priority correction). In the current build the `fall` verdict is reachable *only* through the descent-speed trigger, while the lying predicate — which encodes the angle and ground-proximity conditions — can escalate no further than `warning`. Two failure modes follow: a rapid head movement close to the camera can register a `fall` while the subject is upright, and a subject already motionless on the floor generates no velocity spike and therefore never reaches `fall`. FR-3.1 states the *intended* behaviour and is retained unchanged; the implementation must be corrected to satisfy it. This is the first scheduled task of FYP-2.

3.4 FR-4 — Interactive Monitoring Dashboard

ID	Requirement	Priority	Verif.	Acceptance condition
FR-4.1	The system **shall** render a heads-up display on the video stream showing the current status (colour-coded green / amber / red), confidence, a telemetry panel with body angle, drop speed and a stability bar, and a frame-rate counter.	MUST	D	All HUD elements are present and update continuously.
FR-4.2	The system **shall** serve a web dashboard presenting a live MJPEG video feed, a status card with confidence bar, telemetry tiles (persons, body angle, drop speed, cumulative falls), and an incident list.	MUST	D	Dashboard loads at `http://127.0.0.1:5000` and displays all panels.
FR-4.3	The dashboard **shall** refresh all telemetry at intervals not exceeding 1 second (implemented at 500 ms).	MUST	T	Values visibly track subject movement without manual refresh.
FR-4.4	The dashboard **should** provide a browsable, filterable history of past incidents with associated capture thumbnails.	WON'T (FYP-2)	D	Navigation entry is present but explicitly disabled and labelled as future scope.
FR-4.5	The dashboard **shall** display live system health indicators reflecting actual state: server reachability, and detection status distinguishing *Active*, *Paused* and *No Camera*.	MUST	D, T	Disconnecting the camera changes the indicator to *No Camera* within one poll interval.
FR-4.6	The dashboard **shall** present a prominent visual overlay when status becomes `fall`.	MUST	D	Overlay appears on fall and clears on return to a lower severity.

NEW IN V3 **FR-4.5** formalises a requirement that was absent from V2. The Checkpoint-1 dashboard displayed permanently hard-coded "Online" and "Active" indicators that did not reflect reality — they continued to read "Active" when the camera was disconnected. Health indicators are now required to be state-derived.

3.5 FR-5 — Alerting and Notification

ID	Requirement	Priority	Verif.	Acceptance condition
FR-5.1	On detecting a fall the system **shall** emit an audible alarm — a three-tone pattern of 1000 Hz, 1500 Hz and 1000 Hz, each of 200 ms — on the host machine.	MUST	D	Buzzer is audible on fall detection.
FR-5.2	The system **shall** display a high-priority alert window on the host machine for a configurable duration (default 3000 ms), dismissing it automatically without operator interaction.	MUST	D	Popup appears and self-dismisses after the configured interval.
FR-5.3	The system **shall** raise a native operating-system notification on fall or warning, so that alerts remain visible when the dashboard is closed or the operator is using another application.	MUST	D	OS notification appears with the dashboard closed and another application in focus.
FR-5.4	The system **may** transmit an HTML e-mail notification containing event details and the captured JPEG as an attachment, over SMTP with STARTTLS.	COULD	T	Implemented; disabled by default pending credential configuration.
FR-5.5	The system **shall** enforce a configurable cooldown period (default 30 seconds) applied collectively across all alert channels, to prevent notification flooding during a sustained event.	MUST	T	Repeated detections within the window produce exactly one alert.
FR-5.6	The system **shall** write every alert to a daily JSON log file `logs/fall_events_YYYYMMDD.json` recording status, confidence, ISO-8601 timestamp and person identifier, and **shall** save one JPEG frame per alert as evidence.	MUST	I	Log file and corresponding capture exist after a detection.
FR-5.7	All blocking alert channels **shall** execute asynchronously so that no notification delays frame processing.	MUST	A, I	Frame rate shows no measurable drop while an alert is being delivered.
FR-5.8	On start-up the system **shall** reload the current day's existing log file so that restarting the application preserves rather than truncates the day's event history.	SHOULD	T	Event count is retained across a restart within the same day.

NEW IN V3 **FR-5.3** (OS notification) and **FR-5.7** (asynchronous delivery) were implemented during Checkpoint-2 and were not present in V2. FR-5.7 arose from a measured defect: the buzzer blocks its calling thread for approximately 600 ms, which visibly stalled the video pipeline on every alert when executed inline.

3.6 FR-6 — Operator Control and System Management

NEW GROUP IN V3 V2 specified no operator controls at all. Live demonstration made clear that an operator must be able to intervene without terminating monitoring.

ID	Requirement	Priority	Verif.	Acceptance condition
FR-6.1	The operator **shall** be able to pause and resume the live video feed from the dashboard, with a clear visual indication of the paused state.	SHOULD	D	Pausing halts the stream and displays a paused overlay; resuming restores it.
FR-6.2	The operator **shall** be able to mute and unmute alerts independently of pausing. Muting **shall** suppress the buzzer, popup and OS notification while **continuing** event logging and dashboard updates.	MUST	D, T	With alerts muted, a detection produces no audible or popup output but does append a log record.
FR-6.3	The system **shall** provide an endpoint to reset tracking state — clearing the feature buffer and smoothing history — without restarting the process.	SHOULD	T	`POST /reset` returns success and detection recommences from a clean state.
FR-6.4	All operational thresholds, paths and hyper-parameters **shall** be defined in a single centralised configuration module, with no threshold values embedded in algorithm code.	MUST	I	Inspection confirms every threshold resolves to a named configuration field.
FR-6.5	The system **should** allow thresholds to be adjusted from the dashboard at runtime.	WON'T (FYP-2)	D	Settings navigation entry is present but explicitly disabled.
FR-6.6	The system **shall** resolve all configured relative paths to absolute paths at start-up, so behaviour is identical regardless of the working directory from which it is launched.	SHOULD	T	Launching from a different directory produces identical log and model resolution.

3.7 FR-7 — Model Training and Data Management

> NEW GROUP IN V3 The training subsystem exists in the codebase but was entirely undocumented in V2, leaving a substantial part of the delivered work unspecified.

ID	Requirement	Priority	Verif.	Acceptance condition
FR-7.1	The system **shall** provide an offline pipeline that extracts frames from labelled videos, computes features using the **same** pose estimation and feature extraction components used at inference, and emits training sequences.	MUST	I	Training imports the same classes as the live path, guaranteeing train/serve feature parity.
FR-7.2	The system **shall** support the three-class label scheme `normal` = 0, `warning` = 1, `fall` = 2.	MUST	I	Label mapping is defined in one place and used consistently.
FR-7.3	Training **shall** apply class weighting (default `[1.0, 2.0, 3.0]`) to compensate for the natural scarcity of fall examples, penalising missed falls most heavily.	MUST	I	Weighted loss is configured and applied.
FR-7.4	Training **shall** implement early stopping with configurable patience and **shall** retain the best-validation checkpoint separately from the final-epoch checkpoint.	SHOULD	I	`fall_detector_best.pth` is produced and auto-discovered at inference.
FR-7.5	The system **shall** support selecting among LSTM, CNN-LSTM and Transformer architectures by configuration value alone, without code modification.	SHOULD	T	Each architecture instantiates and trains via the factory function.
FR-7.6	The system **should** support exporting the trained model to ONNX for optimised edge runtimes.	COULD	T	Export produces a valid opset-13 graph with a dynamic batch axis.
FR-7.7	The system **should** provide synthetic data generation so the full pipeline can be exercised end to end without a labelled video corpus.	SHOULD	D	`python main.py demo` trains and runs detection on a clean machine.
FR-7.8	The system **shall** report per-class precision, recall and F1 together with a confusion matrix when evaluating a trained model.	WON'T (FYP-2)	T	Requires the labelled dataset scheduled for FYP-2.

4. External Interface Requirements

4.1 User Interfaces

The web dashboard shall be responsive and cross-browser compatible, built with HTML5 for structure, CSS3 for presentation (Inter and Outfit typefaces, dark theme, card-based layout), and vanilla JavaScript for status polling and DOM updates. No JavaScript framework or build step is used, so the interface is directly inspectable.

Element	Requirement
Status legibility	Current status shall be readable at a glance from across a room, using both text and colour coding (green / amber / red).
Colour independence	Status shall never be conveyed by colour alone; a textual label shall always accompany it.
Control affordances	Pause and Mute controls shall be adjacent to the video feed and clearly indicate their current state.
Unavailable features	Navigation entries for features deferred to FYP-2 shall be visibly disabled and labelled as such, rather than presenting non-functional controls.
No installation	The dashboard shall require no client-side installation beyond a web browser.

4.2 Hardware Interfaces

Interface	Specification
Camera	Any device enumerable by OpenCV — integrated webcam, USB camera, or IP/RTSP stream by URL. Default source index 0, requested at 640 × 480 @ 30 FPS.
Audio output	System default audio device. No dedicated sound hardware or external audio file is required.
Compute	x86-64 CPU minimum; CUDA-capable GPU detected and used automatically when available.
Display	Required on the host machine for the popup alert window; the dashboard itself may be viewed from any networked device.

4.3 Software Interfaces

Component	Version	Purpose
Python	≥ 3.8	Runtime environment.
Ultralytics (YOLOv8)	≥ 8.0.0	Pose estimation. **Required** — omitted from the V2 dependency list, which prevented installation from succeeding.
PyTorch	≥ 2.0.0	Temporal model execution and training.
OpenCV	≥ 4.8.0	Frame capture, JPEG encoding, overlay drawing.
Flask	≥ 3.0.0	Web server and REST endpoints.
NumPy	≥ 1.24.0	Array and vector mathematics.
scikit-learn	≥ 1.3.0	Training metrics and data splitting.
plyer	≥ 2.1.0	Cross-platform native OS notifications.
ONNX Runtime	≥ 1.16.0	Optional optimised inference runtime.

> **Dependency corrections applied since V2.** The V2 dependency list named `mediapipe`, which is no longer imported anywhere in the codebase, and `smtplib-attachment`, which is not a real installable package — its presence caused `pip install -r requirements.txt` to fail outright. Conversely `ultralytics`, on which pose estimation entirely depends, was absent. All three issues are corrected.

4.4 Communication Interfaces — REST API

The system shall expose the following HTTP interface on port 5000.

Method	Route	Purpose	Response
GET	`/`	Serve the dashboard application.	`text/html`
GET	`/static/<path>`	Serve CSS and JavaScript assets.	Static file
GET	`/video_feed`	Continuous annotated video stream with skeleton and HUD.	`multipart/x-mixed-replace`
GET	`/health`	Liveness probe; reports whether the neural checkpoint loaded.	`{status, model_loaded, timestamp}`
GET	`/status`	Primary telemetry endpoint polled by the dashboard.	`{current_status, event_summary, camera_connected, alerts_muted}`
POST	`/detect`	Stateless single-frame inference (form-data file or base64 JSON).	`{status, confidence, keypoints_detected, processing_time_ms}`
POST	`/detect/batch`	Sequence inference over an ordered list of base64 frames.	`{results[], final_status, final_confidence, num_frames, processing_time_ms}`
POST	`/alerts/mute` NEW	Toggle or set alert muting. Empty body toggles; `{"muted": bool}` sets explicitly.	`{muted}`
POST	`/reset`	Clear feature buffer and smoothing history.	`{message}`

Representative `/status` response

```
{
  "current_status": {
    "status":       "normal",     // normal | warning | fall
    "confidence":   100.0,        // percentage
    "person_count": 1,
    "angle":        87.5,         // degrees  — explainability evidence
    "speed":        0.0           // px/frame — explainability evidence
  },
  "event_summary": {
    "total_events": 3,
    "falls":        1,
    "warnings":     2,
    "log_file":     "logs/fall_events_20260726.json"
  },
  "camera_connected": true,       // NEW in V3
  "alerts_muted":     false       // NEW in V3
}
```

All error conditions shall return an `{"error": "..."}` body with an appropriate HTTP status code (400 for malformed input, 500 for processing failure).

5. Non-Functional Requirements

5.1 NFR-P — Performance

ID	Requirement	Priority	Verif.	Target
NFR-P.1	Pose estimation and keypoint extraction shall complete within 25 ms per frame on a standard CPU.	MUST	A	≤ 25 ms
NFR-P.2	Temporal model forward pass shall complete within 8 ms per sequence.	SHOULD	A	≤ 8 ms
NFR-P.3	The visualisation pipeline shall sustain at least 25 FPS end to end with one subject in frame.	MUST	A, D	≥ 25 FPS Met: 25–30 observed
NFR-P.4	A fall shall be recognised and alerted within 2 seconds of occurring.	MUST	T	≤ 2 s
NFR-P.5	Alert delivery shall not measurably reduce the frame rate of the video pipeline.	MUST	A	No visible stall
NFR-P.6	The dashboard shall remain responsive to API requests while the video stream is active.	MUST	T	`/status` responds under concurrent streaming

5.2 NFR-R — Reliability and Availability

ID	Requirement	Priority	Verif.	Acceptance condition
NFR-R.1	Where the neural checkpoint is missing or fails to load, the system shall continue operating on rule-based detection without an uncaught exception.	MUST	T	Renaming the checkpoint yields a warning and continued service.
NFR-R.2	Where camera connectivity drops, the system shall attempt reconnection and resume streaming automatically once the device is available, without terminating the stream or requiring a restart.	MUST	T	Disconnecting and reconnecting the camera restores the feed unattended.
NFR-R.3	The camera handle shall be released whenever the stream ends, including on client disconnect, so the device can always be reacquired.	MUST	I, T	Pause followed by resume reliably reopens the device.
NFR-R.4	Failure of any single alert channel shall not prevent the remaining channels from delivering.	MUST	T	Disabling audio still yields popup, OS notification and log entry.
NFR-R.5	A corrupted or unreadable event log shall not prevent start-up.	SHOULD	T	Malformed JSON is handled and a fresh log begun.
NFR-R.6	The system shall be capable of continuous unattended operation for a full care shift (minimum 8 hours).	SHOULD	T	No memory growth or degradation over an 8-hour run.

5.3 NFR-S — Safety

ID	Requirement	Priority	Verif.	Acceptance condition
NFR-S.1	Decision logic shall bias toward escalation: where the interpretable rules indicate greater severity than the neural network with high confidence, the more severe verdict shall prevail.	MUST	I	Fusion logic implements the severity override.
NFR-S.2	Every classification shall be traceable to the raw biomechanical values that produced it, to support clinical audit.	MUST	D, I	Angle and speed exposed on HUD and API.
NFR-S.3	Muting alerts shall never suppress event logging, so the evidentiary record remains complete.	MUST	T	Muted detection still writes a log record.
NFR-S.4	The system shall present no diagnostic or medical claim; it is an assistive alerting tool only.	MUST	I	No clinical claim appears in the interface or documentation.

5.4 NFR-SEC — Security and Privacy

ID	Requirement	Priority	Verif.	Acceptance condition
NFR-SEC.1	Video streams shall be processed in local memory and shall never be written to disk as continuous recordings.	MUST	I	No video file is produced during operation.
NFR-SEC.2	Only a single JPEG frame per confirmed alert shall be persisted, stored locally in the log directory.	MUST	I	Capture count equals alert count.
NFR-SEC.3	All inference shall execute locally; no video data shall be transmitted to any third-party service.	MUST	I	No outbound video traffic.
NFR-SEC.4	E-mail alerting shall be disabled by default and shall transmit only event metadata and a single frame when enabled.	MUST	I	Default configuration has e-mail off.
NFR-SEC.5	The system shall not perform biometric identification or associate detections with named individuals.	MUST	I	No identity data is stored.
NFR-SEC.6	The dashboard and API shall require authentication, and shall not be exposed on all network interfaces by default.	WON'T (FYP-2)	T	**Not met.** The server currently binds `0.0.0.0` without authentication — acceptable for a controlled demonstration, but a documented prerequisite for real deployment.

5.5 NFR-U — Usability

ID	Requirement	Priority	Verif.	Acceptance condition
NFR-U.1	A non-technical caregiver shall be able to interpret system status without training or documentation.	MUST	D	Status is a plain word plus colour, not a numeric code.
NFR-U.2	Alerts shall be perceivable through at least two independent sensory channels (audible and visual).	MUST	D	Buzzer plus popup, OS notification and dashboard overlay.
NFR-U.3	An operator shall be able to silence an active alert in a single interaction.	SHOULD	D	One click on Mute.
NFR-U.4	The dashboard shall require no client installation or configuration.	MUST	D	Opening the URL is sufficient.

5.6 NFR-M — Maintainability and Portability

ID	Requirement	Priority	Verif.	Acceptance condition
NFR-M.1	Each pipeline stage shall be an independently replaceable module with a narrow interface.	MUST	I	The MediaPipe → YOLOv8 migration touched only the pose module.
NFR-M.2	No threshold or tunable constant shall be embedded in algorithm code.	MUST	I	All values resolve to configuration fields.
NFR-M.3	Training and inference shall share identical feature computation code to prevent train/serve skew.	MUST	I	Both import the same extractor classes.
NFR-M.4	The system shall run on Windows, Linux and macOS, degrading platform-specific features gracefully.	SHOULD	T	Buzzer falls back to terminal bells off Windows.
NFR-M.5	The codebase shall have automated unit tests covering the geometric functions, state machine and fusion logic.	WON'T (FYP-2)	T	**Not met.** No test suite currently exists.

6. Requirement Prioritisation Summary

Priority	Count	Requirements
MUST	38	Core detection pipeline (FR-1.1–1.5, FR-2.1–2.6), decision engine (FR-3.1–3.7), dashboard (FR-4.1–4.3, 4.5, 4.6), primary alerting (FR-5.1–5.3, 5.5–5.7), mute and configuration (FR-6.2, 6.4), training fundamentals (FR-7.1–7.3), and all MUST-level non-functional requirements.
SHOULD	14	FR-1.4, FR-2.7, FR-2.8, FR-5.8, FR-6.1, FR-6.3, FR-6.6, FR-7.4, FR-7.5, FR-7.7, NFR-P.2, NFR-R.5, NFR-R.6, NFR-U.3, NFR-M.4.
COULD	2	FR-5.4 (e-mail alerting), FR-7.6 (ONNX export). Both implemented but not required for demonstration.
WON'T (FYP-2)	6	FR-1.6 multi-person integration, FR-4.4 event history UI, FR-6.5 runtime configuration UI, FR-7.8 evaluation metrics, NFR-SEC.6 authentication, NFR-M.5 automated tests.

7. Verification and Acceptance Criteria

The Checkpoint-2 prototype is considered acceptable when the following criteria are demonstrated together in a single session.

AC	Criterion	Verifies	Evidence artefact
AC-1	Application starts and the dashboard loads without error; `/health` reports the model loaded.	FR-3.2, FR-4.2	Screenshot, health response
AC-2	Live video displays with skeleton overlay tracking a moving subject.	FR-1.1, FR-1.4	Screenshot, demo video
AC-3	Telemetry (persons, angle, speed) updates continuously and plausibly as the subject moves.	FR-3.7, FR-4.3	Demo video
AC-4	A simulated fall produces a `fall` status with confidence above threshold.	FR-3.1, FR-3.6	Screenshot, demo video
AC-5	The fall triggers buzzer, popup and OS notification, the last visible with the dashboard closed.	FR-5.1–5.3	Demo video
AC-6	A JSON event record and a JPEG capture are written for the event.	FR-5.6	Log file, capture image
AC-7	Repeated detections within 30 seconds yield exactly one alert.	FR-5.5	Log inspection
AC-8	Muting suppresses buzzer, popup and OS notification while logging continues.	FR-6.2, NFR-S.3	Demo video, log file
AC-9	Pausing halts the feed and displays the paused state; resuming restores it.	FR-6.1	Demo video
AC-10	Disconnecting the camera changes the indicator to *No Camera*; reconnecting restores the feed unattended.	FR-4.5, NFR-R.2	Demo video
AC-11	Removing the checkpoint leaves the system running on rules alone.	FR-3.4, NFR-R.1	Console output
AC-12	Sustained frame rate of at least 25 FPS is observed on the HUD counter.	NFR-P.3	Screenshot

8. Requirements Traceability

Forward traceability from each requirement group to its realising design section and current build status.

Requirement	Implementing component	SDD section	Status in Checkpoint-2 build
FR-1.1–1.3	`pose_estimator.py`	§5.2.1, §6.1	Implemented
FR-1.4	`_draw_custom_skeleton()`	§5.2.1	Implemented
FR-1.5	`_handle_lost_tracking()`	§5.2.1	Implemented
FR-1.6	`MultiPersonPoseEstimator`	§5.2.2	Standalone only — not integrated
FR-2.1–2.6	`feature_extractor.py`	§5.3, §6.2, §9.1–9.3	Implemented
FR-2.7	`TemporalFeatureProcessor`	§5.3	Training path only
FR-2.8	— (new requirement)	§12.2	Not implemented
FR-3.1	`get_rule_based_status()`	§8.1, §9.4	Partial — fall rule omits angle/ground confirmation
FR-3.2	`FallDetectorCNNLSTM`	§5.4	Implemented
FR-3.3	`_fuse_predictions()`	§9.5	Implemented
FR-3.4	`_load_model()` fallback	§5.5	Implemented
FR-3.5	`_smooth_predictions()`	§9.6	Implemented
FR-3.6–3.7	`process_frame()`, `_draw_hud()`	§5.5	Implemented
FR-4.1–4.3	`_draw_hud()`, `static/`	§5.5, §7.2	Implemented
FR-4.4	— (deferred)	§12.3	Deferred to FYP-2
FR-4.5–4.6	`app.js`, `/status`	§7.1, §7.2	Implemented
FR-5.1–5.3	`alert_system.py`	§5.6	Implemented
FR-5.4	`_email_alert()`	§5.6	Implemented, disabled by default
FR-5.5–5.8	`trigger_alert()`, `_log_event()`	§5.6, §6.4, §8.3	Implemented
FR-6.1–6.3	`app.js`, `/alerts/mute`, `/reset`	§7.1, §7.2	Implemented
FR-6.4, 6.6	`config.py`	§5.1	Implemented
FR-6.5	— (deferred)	§12.3	Deferred to FYP-2
FR-7.1–7.7	`train.py`, `dataset.py`	§5.8	Implemented
FR-7.8	— (deferred)	§12.3	Requires labelled dataset
NFR-P.*	Pipeline design, threading	§10.1	Met
NFR-R.*	Fallbacks, reconnection	§10.2	Met

NFR-S.*	Fusion override, XAI telemetry	§9.5, §5.5	**Met**
NFR-SEC.1–5	No-recording design	§10.3	**Met**
NFR-SEC.6	— (deferred)	§10.3	**Deferred to FYP-2**
NFR-U.*	Dashboard design	§7.2	**Met**
NFR-M.1–4	Modular architecture	§10.4	**Met**
NFR-M.5	— (deferred)	§12.2	**Deferred to FYP-2**

9. Change Log — V2 to V3

This section itemises every substantive change from SRS v2, so the supervisor can review the revision without re-reading the whole document.

9.1 Factual Corrections

#	Issue in V2	Correction in V3
1	**Temporal window duration contradiction.** V2 stated the 30-frame sequence represented "2 seconds" in two places, but "1 second at 30 FPS" in FR-2.4.	Standardised to **approximately one second at 30 FPS**, which is arithmetically correct. Corrected in Section 2.3, FR-2.4 and Appendix A.
2	**MediaPipe / YOLOv8 ambiguity.** V2 named MediaPipe in the glossary and reference list while specifying YOLOv8-Pose in the requirements, leaving the actual technology unclear.	YOLOv8-Pose is stated unambiguously as the pose technology. MediaPipe is retained in the glossary and references but explicitly marked *historical — superseded*, with the migration rationale recorded.
3	**Corrupted inline references.** Text such as "using pose_estimator.py (pose_estimator.py." appeared throughout Section 1.2 with unclosed parentheses and duplicated filenames.	All malformed references rewritten as clean prose.
4	**Unrendered diagram source.** Section 2.1 contained a raw Mermaid code block that appeared in the document as an unreadable single line of text, so the architecture was effectively undocumented.	Replaced with two properly rendered vector diagrams: a system context diagram (Figure 2.1) and a use-case model (Figure 2.2).
5	**Overstated multi-person support.** V2 presented tracking of 5 persons as a delivered function.	Reclassified as FR-1.6 with priority WON'T (FYP-2), with an explicit note that only the single-subject path is integrated.
6	**Incomplete dependency list.** V2 named `mediapipe` (unused) and `smtplib-attachment` (not a real package, causing installation to fail), while omitting `ultralytics`, on which the system depends.	Dependency table in Section 4.3 corrected, with the defect explained.

9.2 Structural Improvements

#	Addition	Purpose
7	Requirement identifiers, MoSCoW priority and verification method on every requirement	Makes each requirement individually addressable, testable and prioritised — the principal weakness of V2, in which requirements were prose paragraphs with no acceptance condition.
8	System context diagram (Figure 2.1)	Defines the system boundary and every external entity, which V2 never stated.
9	Use-case model (Figure 2.2) with an expanded primary use case	Connects requirements to actor goals rather than listing capabilities in isolation.
10	Section 2.6 Operating Environment	V2 scattered environment assumptions across several sections.
11	Section 6 Prioritisation Summary	Gives an at-a-glance view of committed versus deferred scope.
12	Section 7 Acceptance Criteria (AC-1 to AC-12)	Defines objectively what a successful demonstration looks like, with the evidence artefact for each.
13	Section 8 Traceability	Links every requirement forward to its design section and honest build status.
14	Document control: revision history, approval block, related documents	Standard configuration-management practice, absent from V2.

9.3 New Requirements Introduced

ID	Requirement	Reason for addition
FR-2.8	Keypoint-confidence gating for the orientation angle	Raised in response to observed false `warning` states when hips are near the frame edge and YOLO emits low-confidence estimated positions that are consumed unchecked.
FR-4.5	Live system-health indicators	The Checkpoint-1 dashboard hard-coded "Online" and "Active", continuing to display "Active" with the camera disconnected — actively misleading in a safety context.
FR-5.3	Native OS notification channel	Alerts were invisible when the operator was working in another application or had the dashboard closed.
FR-5.7	Asynchronous alert delivery	The buzzer blocks its thread for roughly 600 ms; executed inline it visibly stalled the video pipeline on every alert.
FR-5.8	Log continuity across restart	Prevents the day's audit trail being truncated by a restart.
FR-6.1–6.6	Operator control group	V2 specified no operator controls whatsoever; demonstration showed pause, mute and reset to be operationally necessary.
FR-7.1–7.8	Training and data management group	The training subsystem existed in code but was entirely unspecified, leaving a substantial deliverable undocumented.
NFR-U.*, NFR-M.*	Usability, maintainability and portability categories	V2 covered only performance, reliability, safety and security.

9.4 Defects Recorded Against Existing Requirements

Against	Defect and disposition
FR-3.1	**Fall rule does not implement its specified condition.** The requirement mandates that a fall be confirmed by ground proximity and an angle below 50° following rapid descent; the implementation triggers on descent speed alone. The requirement is *correct as written* and is retained unchanged — the implementation must be brought into conformance. Scheduled as the first task of FYP-2 and recorded in SDD §12.2.
FR-2.7	Feature standardisation is applied during training but not at live inference, creating potential train/serve skew. Reclassified as partially met.

> **On recording defects in a requirements document.** Requirements state intended behaviour, and remain valid even where the current build does not yet satisfy them. Recording the gap explicitly — rather than silently weakening the requirement to match the code — keeps the specification honest and gives FYP-2 a precise, prioritised work list.

Appendix A — Academic Viva and Defence Supplement

Retained from V2 and revised for factual accuracy.

A.1 Why fuse rules with deep learning rather than relying on the network alone?

Neural networks are black-box models. Fusing interpretable biomechanical rules with the network's output makes the system *explainable*. An examiner or clinician can see that a fall was raised because the body angle fell to 35° at a velocity of 22 px/frame and the subject remained down — corroborated by the CNN-LSTM classification. The rules additionally act as a safety guardrail: where they detect greater severity with high confidence, they override the network, because a missed fall is far costlier than a false alarm.

A.2 How does the system avoid false positives from ordinary fast movements?

Three independent mechanisms operate together:

1. **Temporal sequence buffering.** A 30-frame window (approximately one second) is analysed, so a rapid downward movement immediately followed by an upright posture is classified as normal.
2. **Counter-based hysteresis.** The fall counter must reach 2 before a fall is even a candidate, and it decays rather than resetting, so isolated noisy frames dissipate.
3. **Temporal smoothing.** A fall must be classified in at least 3 of the last 5 frames before it is emitted.

Honest qualification. These mechanisms are effective against transient noise but do not fully resolve the FR-3.1 defect described in Section 9.4: because the fall trigger is currently driven by descent speed alone, a sufficiently sustained rapid head movement can still produce a false positive. The corrective work is specified and scheduled.

A.3 Trade-offs among the three model architectures

Architecture	Assessment
LSTM	Captures temporal ordering well, but treats the 51 input dimensions as an unstructured vector and so models spatial relationships between joints inefficiently.
CNN-LSTM (chosen)	The 1-D convolutional front-end learns local joint-motion patterns automatically before the LSTM models longer-range progression. Best accuracy-per-millisecond balance for edge hardware.
Transformer	Strongest long-range modelling through self-attention, but higher compute cost and a greater tendency to overfit the modest keypoint dataset available at FYP-1 scale.

A.4 Why YOLOv8-Pose instead of MediaPipe?

MediaPipe was evaluated during Checkpoint-1 and provides excellent single-subject tracking with very low latency. It was replaced because it is fundamentally single-person oriented, whereas the target deployment — a care ward or shared living space — routinely contains several people, and because YOLOv8-Pose proved more robust to partial occlusion, which is common when furniture obscures part of a subject. YOLOv8 also detects persons and regresses keypoints in a single pass, simplifying the pipeline. The cost is a larger model and higher CPU load, mitigated by selecting the nano variant.

A.5 What happens if the model file is missing?

The system logs a warning and continues using rule-based detection alone. This is a deliberate reliability property (NFR-R.1): an unattended safety system must degrade rather than fail, and it is what allows the prototype to be demonstrated on a machine that has never run training.

Appendix B — Threshold and Parameter Glossary

Parameter	Value	Meaning
Detection confidence	0.5	Minimum YOLO score for a person detection to be accepted.
Joint draw confidence	0.4	Minimum keypoint score for a joint to be drawn on the overlay.
Occlusion tolerance	10 frames	Frames for which last-known keypoints are retained before tracking is declared lost.
Sequence length	30 frames	Temporal window, approximately one second at 30 FPS.
Training stride	5 frames	Sliding-window step during offline sequence generation.
Feature dimensions	51	39 raw keypoint values plus 12 engineered slots (5 active, 7 reserved).
Upright angle threshold	70°	Below this, posture is considered unstable (`warning`).
Lying angle threshold	50°	Below this, combined with ground proximity, posture is considered lying.
Ground proximity	60% height	Head below this fraction of frame height indicates floor level (y > 288 of 480).
Drop speed threshold	20 px/frame	Descent rate indicating rapid, uncontrolled motion.
Fall counter threshold	2 frames	Consecutive qualifying frames before a fall is a candidate.
Lying counter threshold	5 frames	Sustained lying frames before a warning is raised.
Smoothing window	5 frames	Voting window; 3 of 5 for fall, 2 of 5 for warning.
Neural fusion weight	0.7	Weight given to the network verdict; rules receive 0.3.
Rule override confidence	0.8	Rule confidence above which greater severity overrides the network.
Alert confidence gate	0.7	Minimum fused confidence required to raise an alert.
Alert cooldown	30 s	Minimum interval between alerts, applied across all channels.
Popup duration	3000 ms	On-screen alert window lifetime.
Dashboard poll interval	500 ms	Telemetry refresh rate.
Camera reconnect threshold	15 failures	Consecutive read failures before the device is released and reopened.

End of Software Requirements Specification

GuardianAI — Real-Time Automated Fall Detection and Alert System · FYP-2026-G6 · Version 3.0

Supersedes SRS v2 · Prepared for FYP-I Checkpoint-2 · Subject to Faculty Supervisor review and approval